

RAPPORT DE TP | MASTER 1 SII

Systemes Multi-Agents et Planification

Projet JADE/Python : enchères ACL, agent mobile, AIMA POP, synchronisation Bill/Tom et sorties vérifiées.

BINÔME

BOUSSELOUB Islem

212131072965

CHEMLAL Mohamed Mounir

222231502110

PARTIES RÉALISÉES

Enchère JADE • Agent mobile multicritère
AIMA POP • Planification distribuée Bill/Tom

Table des matières

Introduction	1
1 Cadre technique	2
1.1 Architecture du dépôt	2
1.2 Outils utilisés	2
1.3 Commandes de validation	2
2 Partie 1 – Enchère JADE	3
2.1 Objectif	3
2.2 Structure des classes	3
2.3 Trace console : scénario vendu	3
2.4 Trace console : scénario non vendu	4
3 Partie 2 – Agent mobile et multicritère	5
3.1 Score multicritère	5
3.2 Trace console : migration inter-conteneurs	5
3.3 Limite inter-plateformes	6
4 Partie 3 – Test AIMA POP	7
4.1 Exécution	7
5 Partie 4 – Bill/Tom : planification distribuée	8
5.1 Plan et contrainte	8
5.2 Synchronisation	8
5.3 Trace console	8
Conclusion	9

Introduction

Ce rapport présente le mini-projet de Technologies des Agents. Le travail demandé est découpé en quatre parties : une enchère JADE, un agent mobile avec décision multicritère, un test du planificateur POP d'AIMA et une simulation de planification distribuée Bill/Tom. Les outils utilisés sont JADE pour les agents Java, AIMA-Python pour la partie POP, et Python pour la simulation Bill/Tom.

Principe du rapport

Le rapport ne redéfinit pas les notions vues en cours. Il montre l'implémentation réalisée, les paramètres utilisés et les sorties console qui prouvent l'exécution.

Implémentation

Java JADE pour les parties 1 et 2 ; Python pour les parties 3 et 4.

Exécution

Les traces incluses proviennent des scripts exécutés localement.

Validation

Deux scénarios d'enchère, migration inter-conteneurs, POP AIMA et simulation Bill/Tom.

Outils

JADE 4.6.0, Java, AIMA-Python, LuaLaTeX.

Chaque affirmation importante est donc liée à une trace : vendu ou non vendu pour l'enchère, offres collectées pour l'agent mobile, plan retourné par AIMA et état final atteint pour Bill/Tom.

Cadre technique

1.1 Architecture du dépôt

Le dépôt du mini-projet est situé dans :

```
C:\Users\DELL\Downloads\mini_projet_techAg-20260515T084328Z-3-001\mini_projet_techAg
```

Les dossiers utiles sont :

```
part1\negotiation    Enchere JADE
part2                Agent mobile et score multicritere
part3                Test AIMA Partial Order Planner
part4                Simulation Bill/Tom
```

1.2 Outils utilisés

Outil	Partie	Rôle
Java / JADE	1 et 2	Création des agents, messages ACL, migration entre conteneurs.
Python AIMA	3	Exécution du planificateur POP existant.
Python	4	Simulation du plan distribué Bill/Tom et vérification de l'état final.
LaTeX	Rapport	Mise en forme et intégration des sorties console.

1.3 Commandes de validation

```
java -cp ".\jade.jar" part1.negotiation.AuctionLauncher sold
java -cp ".\jade.jar" part1.negotiation.AuctionLauncher unsold
java -cp ".\jade.jar" part2.Part2Launcher
python part3\run_planning_partial_order.py
python part4\part4_ap.py --cli
```

Partie 1 – Enchère JADE

JADE

ACL

CFP

PROPOSE

Prix de réserve

2.1 Objectif

Nous avons implémenté une enchère avec un SellerAgent et deux BuyerAgent. Le vendeur envoie un CFP à chaque round. Les acheteurs répondent par PROPOSE si le prix proposé reste sous leur maximum, sinon par REFUSE. Le vendeur garde la meilleure offre et termine par acceptation ou rejet.

Q Point vérifié

Le prix de réserve reste connu uniquement du vendeur. La vente n'est confirmée que si la meilleure offre atteint cette réserve.

2.2 Structure des classes

Classe	Rôle
SellerAgent	Pilote les rounds, filtre les réponses avec conversationId et inReplyTo, vérifie la réserve.
BuyerAgent	Répond aux CFP, propose currentPrice + increment, refuse si le prix dépasse maxPrice.
AuctionLauncher	Lance les scénarios sold et unsold.

2.3 Trace console : scénario vendu

```
[Launcher] Scenario=sold
[Launcher] Product start=100.0, reserve=160.0, buyer1Max=150.0, buyer2Max=200.0
[seller] Round 1 CFP envoye. currentPrice=100.0, replyWith=auction-seller-1778859029687-round-1, activeBuyers=[buyer1, buyer2]
[buyer2] PROPOSE price=120.0 round=1 inReplyTo=auction-seller-1778859029687-round-1
[buyer1] PROPOSE price=120.0 round=1 inReplyTo=auction-seller-1778859029687-round-1
[seller] Meilleure offre actuelle: buyer=buyer1, price=120.0, round=1
[seller] Round 3 CFP envoye. currentPrice=140.0, replyWith=auction-seller-1778859029687-round-3, activeBuyers=[buyer1, buyer2]
[buyer2] PROPOSE price=160.0 round=3 inReplyTo=auction-seller-1778859029687-round-3
[seller] REFUSE recu depuis buyer1
[seller] Round 5 CFP envoye. currentPrice=180.0, replyWith=auction-seller-1778859029687-round-5, activeBuyers=[buyer2]
[buyer2] PROPOSE price=200.0 round=5 inReplyTo=auction-seller-1778859029687-round-5
[seller] Aucune meilleure offre au round 6.
[seller] RESULTAT: VENDU a buyer2 pour 200.0
[buyer2] Proposition acceptee: result=SOLD;winner=buyer2;finalPrice=200.0
[seller] Enchere terminee. Agent conserve dans la table JADE.
[buyer1] Agent conserve dans la table JADE.
[buyer2] Agent conserve dans la table JADE.
```

**Résultat du scénario vendu**

La réserve vaut 160.0 et la meilleure offre atteint 200.0. La trace confirme donc la vente à buyer2.

2.4 Trace console : scénario non vendu

```
[Launcher] Scenario=unsold
[Launcher] Product start=100.0, reserve=300.0, buyer1Max=150.0, buyer2Max=180.0
[seller] Round 1 CFP envoye. currentPrice=100.0, replyWith=auction-seller-1778859033887-round-1, activeBuyers=[buyer1, buyer2]
[buyer1] PROPOSE price=120.0 round=1 inReplyTo=auction-seller-1778859033887-round-1
[buyer2] PROPOSE price=120.0 round=1 inReplyTo=auction-seller-1778859033887-round-1
[seller] Round 4 CFP envoye. currentPrice=160.0, replyWith=auction-seller-1778859033887-round-4, activeBuyers=[buyer2]
[buyer2] PROPOSE price=180.0 round=4 inReplyTo=auction-seller-1778859033887-round-4
[seller] REFUSE recu depuis buyer2
[seller] Aucune meilleure offre au round 5.
[seller] RESULTAT: NON VENDU. Meilleur prix atteint=180.0
[buyer1] Proposition rejetee: result=NOT_SOLD;bestPrice=180.0
[buyer2] Proposition rejetee: result=NOT_SOLD;bestPrice=180.0
[seller] Enchere terminee. Agent conserve dans la table JADE.
[buyer1] Agent conserve dans la table JADE.
[buyer2] Agent conserve dans la table JADE.
```

**Résultat du scénario non vendu**

La réserve vaut 300.0 et le meilleur prix atteint seulement 180.0. Le produit n'est donc pas vendu.

Partie 2 – Agent mobile et multicritère

Agent mobile

doMove

beforeMove

afterMove

Score pondéré

3.1 Score multicritère

Nous avons implémenté l'exercice 5 du DW4 avec trois vendeurs. L'acheteur collecte le prix, la qualité et le coût de livraison, puis applique un score pondéré :

$$score = 0.35 \times priceScore_{min} + 0.50 \times qualityScore_{max} + 0.15 \times deliveryScore_{min}$$

Vendeur	Prix	Qualité	Livraison	Score
seller1	900	8	5	0.4083
seller2	780	6	3	0.5000
seller3	850	9	7	0.6458

3.2 Trace console : migration inter-conteneurs

```
[Launcher-Part2] Mode: acheteur mobile avec migration inter-conteneurs.
[buyerPart2] Demarrage buyer. product=Ordinateur portable, priceWeight=0.35, qualityWeight=0.5, deliveryCostWeight=0.15
[buyerPart2] Mode mobile active. Home=buyer-site, itinerary=Main-Container
[buyerPart2] Migration demandee vers Main-Container
[buyerPart2] beforeMove depuis buyer-site
[buyerPart2] afterMove vers Main-Container
[buyerPart2] CFP local envoye aux vendeurs seller1, seller2, seller3 depuis Main-Container
[buyerPart2] Offre mobile collectee: seller=seller1, price=900.0, quality=8.0, deliveryCost=5.0
[buyerPart2] Offre mobile collectee: seller=seller3, price=850.0, quality=9.0, deliveryCost=7.0
[buyerPart2] Offre mobile collectee: seller=seller2, price=780.0, quality=6.0, deliveryCost=3.0
[buyerPart2] Migration demandee vers buyer-site
[buyerPart2] beforeMove depuis Main-Container
[buyerPart2] afterMove vers buyer-site
[buyerPart2] Retour au conteneur home. Offres collectees=3
[buyerPart2] Score seller1 = 0.4083333333333333 (price=0.0, quality=0.6666666666666666, deliveryCost=0.5)
[buyerPart2] Score seller3 = 0.6458333333333334 (price=0.4166666666666667, quality=1.0, deliveryCost=0.0)
[buyerPart2] Score seller2 = 0.5 (price=1.0, quality=0.0, deliveryCost=1.0)
[buyerPart2] Meilleure offre retenue: seller=seller3, price=850.0, quality=9.0, deliveryCost=7.0
[buyerPart2] Agent conserve dans la table JADE.
[seller1] Agent conserve dans la table JADE.
[seller2] Agent conserve dans la table JADE.
[seller3] Agent conserve dans la table JADE.
```



Résultat de la sélection

seller3 gagne. Son prix n'est pas minimal, mais sa qualité est maximale et ce critère a le poids le plus élevé.

3.3 Limite inter-plateformes

Nous n'avons pas exécuté une vraie migration inter-plateformes complète. Pour la tester réellement, il faut deux plateformes JADE séparées, des adresses MTP accessibles, les mêmes classes déployées sur les deux JVM et une configuration de sécurité compatible. Le code exécuté valide donc le cas inter-conteneurs.

Partie 3 – Test AIMA POP

AIMA

POP

Socks and Shoes

GraphPlan

4.1 Exécution

Cette partie consiste à tester le code AIMA existant. Nous avons exécuté le script local `part3/run_planning_partial_order.py`.

```
=== Partie 3 : AIMA Partial Order Planner ===
Probleme teste: socks_and_shoes
Plan partiellement ordonne par niveaux:
  Niveau 1 : Start
  Niveau 2 : LeftSock, RightSock
  Niveau 3 : RightShoe, LeftShoe
  Niveau 4 : Finish
Verification POP: OK

=== Tests GraphPlan AIMA ===
Spare tire: ['Remove(Spare, Trunk)', 'Remove(Flat, Axle)', 'PutOn(Spare, Axle)']
Three block tower: ['Move(B, Table, C)', 'MoveToTable(C, A)', 'Move(A, Table, B)']
Verification GraphPlan: OK
```



Observation

LeftSock et RightSock sont au même niveau. Les deux actions peuvent donc être faites dans n'importe quel ordre. Les chaussures viennent ensuite, car chaque chaussure dépend de la chaussette correspondante.

Partie 4 – Bill/Tom : planification distribuée

Bill

Tom

S3 < S4

Synchronisation

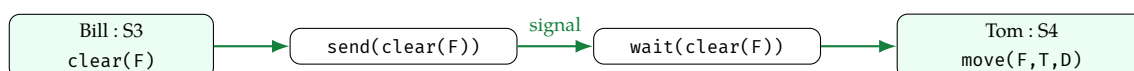
Simulation Python

5.1 Plan et contrainte

Nous avons implémenté l'exemple Bill/Tom sous forme de simulation Python. La contrainte principale est $S3 < S4$: Tom ne peut déplacer F que lorsque Bill a rendu `clear(F)` vrai.

Étape	Agent	Action
S3	Bill	<code>movetotable(E, F)</code>
S2	Bill	<code>move(A, B, E)</code>
S1	Bill	<code>move(B, T, A)</code>
S6	Tom	<code>movetotable(C, D)</code>
S5	Tom	<code>move(D, T, C)</code>
S4	Tom	<code>move(F, T, D)</code>

5.2 Synchronisation



5.3 Trace console

```

=== Partie 4 : planification centralisee pour plans distribues ===
Etat initial: ['clear(A)', 'clear(C)', 'clear(E)', 'on(A,B)', 'on(B,T)', 'on(C,D)', 'on(D,T)', 'on(E,F)', 'on(F,T)']
Bill | S3 | movetotable(E,F) | clear(F) devient vrai
Tom | S6 | movetotable(C,D) | clear(D) devient vrai
Tom | S5 | move(D,T,C) | on(D,C) devient vrai
Bill | SYNC | send(clear(F)) | signal envoie a Tom
Tom | SYNC | wait(clear(F)) | signal recu de Bill
Bill | S2 | move(A,B,E) | on(A,E) devient vrai
Tom | S4 | move(F,T,D) | on(F,D) devient vrai
Bill | S1 | move(B,T,A) | on(B,A) devient vrai
Etat final atteint: ['clear(B)', 'clear(F)', 'on(A,E)', 'on(B,A)', 'on(C,T)', 'on(D,C)', 'on(E,T)', 'on(F,D)']
Verification partie 4 : OK
  
```

Conclusion

Nous avons implémenté et exécuté les quatre parties demandées. L'enchère JADE couvre un cas vendu et un cas non vendu. L'agent mobile collecte trois offres et choisit le vendeur selon un score pondéré. Le script AIMA confirme le fonctionnement du POP sur *socks and shoes*. La simulation Bill/Tom montre une synchronisation explicite autour de `clear(F)`. L'amélioration principale reste le test réel de la migration inter-plateformes sur deux plateformes JADE séparées.